

Bàn về ứng dụng của điều kiện cần

Fang Qi

Nguồn gốc: On the application of necessary conditions

I0I China candidate team papers / 2002 / Fang Qi.pdf

Tóm tắt

Điều kiện cần là một quan hệ logic giữa các mệnh đề. Trong quá trình giải bài, nếu biết dùng tốt điều kiện cần, ta có thể nâng cao hiệu quả thuật toán. Bài viết trước hết nêu định nghĩa của điều kiện cần, sau đó dựa trên đặc điểm của nó để phân tích các ứng dụng trong nhiều dạng bài toán, chỉ ra rằng điểm mấu chốt khi dùng điều kiện cần là “giảm phần dư thừa, thể hiện bản chất”. Cuối cùng, bài viết tổng kết một vài kinh nghiệm khi tìm điều kiện cần.

Từ khóa. Mệnh đề; điều kiện cần; hiệu quả; độ chính xác.

1 Định nghĩa điều kiện cần

Trong toán học, một câu có thể phán định đúng hoặc sai được gọi là một **mệnh đề**. Ta thường dùng các chữ cái Latin thường $p, q, r, s, \dots$ để biểu diễn mệnh đề. Nếu từ mệnh đề p có thể suy ra mệnh đề q , tức là “nếu p đúng thì q đúng”, ta viết

$$p \Rightarrow q.$$

Nói chung, nếu đã biết $p \Rightarrow q$, thì p là **điều kiện đủ** của q , còn q là **điều kiện cần** của p . Ví dụ, $x^2 > 0$ là điều kiện cần của $x > 0$. Ở đây tập nghiệm của $x^2 > 0$ chứa tập nghiệm của $x > 0$:

$$\{x \mid x > 0\} \subseteq \{x \mid x^2 > 0\}.$$

Có thể hình dung bằng một biểu đồ Venn: miền ứng với điều kiện $x > 0$ nằm bên trong miền lớn hơn ứng với điều kiện $x^2 > 0$.

2 Ứng dụng của điều kiện cần

Trong các bài toán tin học, điều kiện cần có phạm vi ứng dụng rất rộng. Bài viết tập trung vào hai mặt chính: thu hẹp phạm vi cần xét và làm lộ ra bản chất của bài toán.

2.1 Thu hẹp phạm vi tìm kiếm

Phần lớn bài toán yêu cầu tìm các phương án hoặc giá trị cụ thể thỏa một số điều kiện. Gọi các điều kiện ấy là mệnh đề p , và gọi tập nghiệm là A . Khi giải bài, dĩ nhiên ta không thể biết ngay nghiệm chính xác

của A . Thông thường ta xuất phát từ điều kiện đã biết q , xét trong một phạm vi lớn hơn B , rồi dùng liệt kê, tìm kiếm, quy hoạch động, tham lam hoặc những phương pháp khác để sàng lọc, cuối cùng tìm ra tập nghiệm A thỏa p .

Trong quá trình ấy, B chứa A , và mọi nghiệm trong A cũng thỏa điều kiện q . Do đó $p \Rightarrow q$, tức q là điều kiện cần của p . Ta bắt đầu từ điều kiện q tương đối “lỏng”, rồi tìm trong B các nghiệm thỏa điều kiện p chặt hơn. Có thể thấy độ lỏng của điều kiện cần ban đầu, tức kích thước của B , ảnh hưởng trực tiếp đến hiệu quả giải bài. Khi q càng gần p , tập B càng gần A , tỉ lệ “vàng” trong hỗn hợp càng cao, phần “cát” cần loại bỏ càng ít, và hiệu quả tự nhiên càng tốt.

Tuy vậy, điều này phải đặt trên cơ sở khả thi. Nếu điều kiện q khó kiểm tra hoặc tốn rất nhiều thời gian xử lý, thì dù q có chính xác đến đâu cũng mất ý nghĩa. Vì vậy, dưới nguyên tắc “dễ cài đặt”, ta nên cố gắng tìm điều kiện cần càng chính xác càng tốt để thu hẹp phạm vi giải và tăng tốc độ tìm nghiệm.

Ví dụ 1: cạnh bắt buộc trong ghép hoàn hảo

Mô tả bài toán. Cho đồ thị hai phía $G(V, E)$. Hỏi những cạnh nào là bắt buộc trong mọi ghép hoàn hảo. Đồ thị có tổng cộng $2n$ đỉnh, với $n \leq 100$.

Phân tích. Gọi tập các cạnh bắt buộc là A . Một cạnh e thuộc A khi và chỉ khi sau khi xóa e , đồ thị còn $g = G - e$ không còn ghép hoàn hảo. Đây có thể xem là điều kiện p . Từ đó ta nhận được thuật toán đầu tiên:

1. Liệt kê mọi cạnh $e \in E$ của đồ thị G .
2. Xóa tạm e khỏi đồ thị, đặt $g = G - e$.
3. Tìm ghép cực đại của g .
4. Nếu g không có ghép hoàn hảo thì $e \in A$, ngược lại thì không.

Dùng thuật toán Hungary để tìm ghép cực đại mất $O(n^2)$, còn số cạnh cần liệt kê có thể là $O(n^2)$. Vì thế độ phức tạp là $O(n^4)$, quá lớn.

Trong thuật toán trên, theo lối nghĩ thông thường, ta vô tình đặt phạm vi ban đầu B bằng toàn bộ tập cạnh E , tức điều kiện cần q chỉ là “cạnh thuộc E ”. Phạm vi tìm kiếm vì vậy bị đặt quá rộng.

Thực ra, nếu $e \in A$, thì e phải thuộc bất kỳ ghép hoàn hảo nào. Nói cách khác, mọi ghép hoàn hảo đều chứa tất cả các cạnh bắt buộc. Ta có thể cải tiến:

1. Trước hết tìm một ghép hoàn hảo M của G .
2. Chỉ liệt kê các cạnh $e \in M$.
3. Với mỗi cạnh ấy, xóa tạm e , tìm ghép cực đại của $G - e$.
4. Nếu $G - e$ không còn ghép hoàn hảo thì $e \in A$, ngược lại thì không.

Ở đây, phạm vi ban đầu được thu hẹp thành một ghép hoàn hảo, và điều kiện cần được làm chính xác thành “cạnh thuộc ghép hoàn hảo M ”. Độ phức tạp giảm còn $O(n^3)$.

Ví dụ này cho thấy điều kiện cần rất hữu ích trong các bài toán tồn tại. Sức mạnh của việc thu hẹp phạm vi còn thể hiện rõ hơn trong tìm kiếm. Những thuật toán như DFS hay BFS không có heuristic thường có độ phức tạp hàm mũ; vì vậy, giảm lượng tìm kiếm là điểm then chốt. Dùng điều kiện cần để cắt bỏ những nhánh không còn hy vọng chính là thao tác thường gọi là **cắt tỉa**.

Ví dụ 2: Puzzle, GDOI 1998

Mô tả bài toán. Xem phụ lục về bài *Puzzle*.

Phân tích. Đây là một bài tìm kiếm điển hình. Theo đề bài, ta biết:

1. cần biến trạng thái ban đầu thành trạng thái đích trong hữu hạn bước, hoặc xác định là vô nghiệm;
2. số bước di chuyển đã biết;
3. số lần di chuyển theo từng hướng đã biết, nhưng thứ tự chưa biết;
4. nghiệm phải đồng thời thỏa các ràng buộc ở hai ý trên.

Ta có thể dùng DFS để lần lượt sinh các dãy di chuyển khả dĩ và in ra dãy thỏa đề. Tìm kiếm toàn bộ như vậy rõ ràng không chấp nhận được về thời gian. Có thể cắt tỉa từ các mặt sau.

Số lần của từng hướng di chuyển. Vì số lần của từng hướng đã biết, nếu trong quá trình tìm kiếm số lần dùng một hướng nào đó vượt giới hạn thì không cần tiếp tục theo hướng ấy.

Vị trí ô trống và số lần của các hướng. Do trạng thái hiện tại, trạng thái đích và số lần còn lại của từng hướng đều đã biết, ta có thể kiểm tra liệu ô trống hiện tại có thể đi đến vị trí ô trống đích bằng tập hướng còn lại hay không. Cụ thể, cần thỏa

$$x_{\text{đích}} - x_{\text{hiện tại}} = \#4_{\text{còn lại}} - \#3_{\text{còn lại}},$$

và

$$y_{\text{đích}} - y_{\text{hiện tại}} = \#2_{\text{còn lại}} - \#1_{\text{còn lại}}.$$

Ở đây các số 1, 2, 3, 4 lần lượt là bốn thao tác trong đề bài.

Vị trí các ký tự và số lần của các hướng. Tương tự, dựa trên trạng thái hiện tại, trạng thái đích và số lần còn lại của các hướng, ta kiểm tra từng ký tự có còn khả năng đi đến vị trí đích hay không.

Trong lập luận trên, đặt p là mệnh đề “dãy thao tác có thể đưa trạng thái đầu về trạng thái cuối”. Gọi tập các dãy thao tác thỏa p là A . Ba phép cắt tỉa tương ứng với các điều kiện q_1, q_2, q_3 , và các tập dãy thao tác thỏa chúng lần lượt là B_1, B_2, B_3 . Khi đó

$$p \Rightarrow q_1, \quad p \Rightarrow q_2, \quad p \Rightarrow q_3,$$

nên

$$A \subseteq B_1 \cap B_2 \cap B_3.$$

Biểu đồ trong bài gốc minh họa A nằm trong phần giao của ba miền B_1, B_2 và B_3 . Ba điều kiện cần cùng tác dụng làm phạm vi tìm kiếm nhỏ và chính xác hơn. Cần chú ý rằng ba phép cắt tỉa này đều dễ cài đặt và không tốn quá nhiều thời gian; nếu không thì sẽ lợi bất cập hại. Sau những tối ưu ấy, lượng tìm kiếm giảm mạnh và chương trình chạy nhanh hơn nhiều.

Ví dụ 3: Musketeers, POI 1999/2001

Mô tả bài toán. n người đứng thành vòng tròn và được đánh số từ 1 đến n . Các trận quyết đấu diễn ra giữa hai người kề nhau; người thắng ở lại trong vòng, người thua bị loại. Không biết thứ tự các trận đấu, chỉ biết tổng cộng có $n - 1$ trận. Cho quan hệ thắng thua giữa mọi cặp người, hỏi những ai có thể là người thắng cuối cùng. $n \leq 100$.

Phân tích. Bài này tương tự bài *Gộp đá* của NOI 1995. Bắt chước bài đó, ta có một cách quy hoạch động.

Gọi $E[i, j]$ là kết quả thắng thua giữa i và j : nếu i thắng j thì $E[i, j] = \text{true}$, nếu j thắng i thì $E[i, j] = \text{false}$. Gọi $A[i, j, k]$ cho biết người k có thể thắng trong đoạn liên tiếp từ i đến j hay không, trong đó k nằm trong đoạn ấy. Khi có thể thì giá trị là true , ngược lại là false . Công thức chuyển trạng thái là

$$A[i, j, k] = \begin{cases} \text{tồn tại } i \leq y \leq x < k \text{ sao cho} \\ \text{true,} & A[i, x, y] \wedge A[x + 1, j, k] \wedge E[k, y], \\ \text{hoặc tồn tại } k < u \leq v \leq j \text{ sao cho} \\ & A[u, j, v] \wedge A[i, u - 1, k] \wedge E[k, v]; \\ \text{false,} & \text{ngược lại.} \end{cases}$$

Thuật toán này có độ phức tạp thời gian $O(n^5)$ và bộ nhớ $O(n^3)$, quá lớn để dùng trực tiếp.

Chú ý rằng điều kiện cần để người số x thắng trong toàn bộ vòng là người ấy có thể “gặp chính mình”. Ta xem vòng như một đoạn thẳng bằng cách tách điểm x thành hai bản sao ở hai đầu; mọi người ở giữa bị loại, còn x không thua. Với cách nhìn này, trong một đoạn liên tiếp chỉ cần xét hai người ở hai đầu có thể thắng để gặp nhau hay không, không cần lưu người thắng ở giữa. Như vậy trạng thái giảm đi một chiều.

Gọi $\text{meet}[i, j]$ cho biết i và j có thể gặp nhau hay không. Công thức chuyển trạng thái là

$$\text{meet}[i, j] = \begin{cases} \text{true,} & \text{tồn tại } i < k < j \text{ sao cho } \text{meet}[i, k] \wedge \text{meet}[k, j] \wedge (E[i, k] \vee E[j, k]); \\ \text{false,} & \text{ngược lại.} \end{cases}$$

Độ phức tạp thời gian giảm còn $O(n^3)$, và bộ nhớ giảm còn $O(n^2)$. Trong bài này, hai cách chọn điều kiện cần về thắng thua dẫn đến hai lối quy hoạch động khác nhau. Cách sau chính xác hơn, giảm tính toán dư thừa và cải thiện cả thời gian lẫn bộ nhớ.

Các ví dụ trên phân tích cách dùng điều kiện cần để thu hẹp phạm vi trong liệt kê, tìm kiếm và quy hoạch động. Ta thấy khi điều kiện cần được chọn chính xác, khác biệt trước và sau tối ưu là rất rõ rệt.

Nếu “thu hẹp phạm vi tìm kiếm” chủ yếu là tối ưu trên thuật toán đã có, thì phần tiếp theo cho thấy việc chọn đúng điều kiện cần còn có thể làm lộ bản chất bài toán, từ đó dẫn ta đến thuật toán mới phản ánh tốt hơn các quan hệ bên trong.

2.2 Làm lộ bản chất của bài toán

Ví dụ 4: Experiment, GDOI 2001

Mô tả bài toán. Xem phụ lục về bài *Experiment*.

Phân tích. Nếu trừu tượng hóa n_1 và n_2 mảnh thiên thạch thành các đỉnh, còn bỏ qua các thông tin khác, bài toán trở thành bài tìm ghép tốt nhất trên đồ thị hai phía. Có thể dùng thuật toán Hungary hoặc luồng chi phí để giải. Độ phức tạp cỡ $O(n_1^2 n_2^2)$; khi n_1, n_2 đều đạt giá trị tối đa 500 thì không thể chịu được.

Thực ra, nếu chỉ xem đây là bài ghép thì ta mới thấy tính phổ biến của mâu thuẫn, nhưng chưa thấy tính đặc thù. Suy nghĩ kỹ hơn, trọng số trên các cạnh của đồ thị hai phía không được gán tùy ý, mà bằng trị tuyệt đối của hiệu khối lượng giữa hai đỉnh được nối. Đây là khác biệt lớn nhất giữa mô hình của bài này và đồ thị hai phía tổng quát.

Bài gốc dùng ba hình nhỏ để minh họa các cách nối khi đặt hai dãy khối lượng trên trục tọa độ. Sau khi sắp xếp khối lượng các mảnh từ sao A và sao B , một điều kiện cần của ghép tối ưu là các cạnh được

chọn không cắt nhau. Trường hợp hai cạnh cắt nhau và không cắt nhau trong một cấu hình đặc biệt có hiệu quả như nhau, nên ta quy ước chỉ chọn phương án không cắt nhau.

Nói cách khác, giả sử một thí nghiệm chọn mảnh có khối lượng a_1 từ sao A và b_1 từ sao B , còn một thí nghiệm khác chọn a_2, b_2 . Để tổng hiệu khối lượng là tốt nhất, không thể tồn tại hai tình huống

$$a_1 < a_2 \text{ và } b_1 > b_2, \quad \text{hoặc} \quad a_1 > a_2 \text{ và } b_1 < b_2.$$

Với điều kiện này, bài toán đồng thời thỏa tính không hậu hiệu và nguyên lý tối ưu, nên có thể giải bằng quy hoạch động. Trước hết sắp xếp hai dãy số từ nhỏ đến lớn. Giả sử $n_1 \leq n_2$; nếu không thì đổi vai trò hai dãy. Mảng $A[i, j]$ biểu diễn hiệu quả tốt nhất khi xét i mảnh đầu của nhóm thứ nhất và j mảnh đầu của nhóm thứ hai. Công thức chuyển là

$$A[i, j] = \begin{cases} \min\{A[i, j-1], A[i-1, j-1] + |w_1[i] - w_2[j]|\}, & j > i, \\ A[i-1, j-1] + |w_1[i] - w_2[j]|, & j = i. \end{cases}$$

Trong đó w_1 lưu khối lượng các mảnh của nhóm thứ nhất, còn w_2 lưu khối lượng các mảnh của nhóm thứ hai.

Thuật toán mới mất $O(n_1 \log n_1 + n_2 \log n_2)$ để sắp xếp và $O(n_1 n_2)$ cho quy hoạch động, nên độ phức tạp tổng thể là $O(n_1 n_2)$. Đây chính là kết quả của việc điều kiện cần đã chạm đúng điểm cốt lõi của bài toán.

3 Cách tìm điều kiện cần

Qua các ví dụ điển hình ở trên, có thể rút ra một vài kinh nghiệm khi tìm điều kiện cần.

Quy nạp cái chung từ đặc thù. Như cách xác định tập B trong ví dụ 1, tính phổ biến thường nằm trong tính đặc thù và biểu hiện thông qua đặc thù. Điều này cũng phù hợp với thói quen tư duy của ta: khi gặp một bài khó, ta thường thử bắt đầu từ những trường hợp đặc biệt đơn giản nhất, dần dần đi sâu hơn, rồi cuối cùng phân tích và quy nạp ra quy luật tổng quát.

Tìm điều kiện cần từ nhiều mặt. Đôi khi, đặc biệt trong thuật toán tìm kiếm, một phép cắt tĩa đơn lẻ là chưa đủ mạnh. Khi các yếu tố bên trong tác động lẫn nhau và khó nắm bắt toàn cục, ta có thể thử phân tích bài toán từ nhiều phía để tìm các điều kiện cần, tách bài toán thành nhiều mặt, rồi tổng hợp các điều kiện cần ấy khi sử dụng. Cách làm này thường tạo ra hiệu quả cộng hưởng; bài *Puzzle* trong ví dụ 2 là một trường hợp điển hình.

So sánh với mô hình ban đầu để phân tích giống và khác. Bản chất thường hiện ra trong so sánh. Ở bài *Experiment*, chính việc so sánh với mô hình đồ thị hai phía tổng quát đã làm lộ ra khác biệt căn bản: điều kiện cần “các cạnh không cắt nhau”. Từ đó ta được dẫn đến lời giải bằng quy hoạch động.

Phương pháp là linh hoạt và đa dạng. Khi tìm điều kiện cần, ta nên giữ nguyên tắc phân tích từng bài toán cụ thể, không bó buộc vào một khuôn mẫu duy nhất.

4 Kết luận

Điều kiện cần thực chất là một cơ sở lý thuyết của suy luận logic, đồng thời cũng là một cách thể hiện quá trình suy nghĩ. Nó có ứng dụng rộng rãi trong nhiều mặt. Khi giải bài, nếu tìm được điều kiện cần

chính xác, ta vừa có thể làm rõ bản chất của bài toán, vừa có thể thu hẹp phạm vi tìm kiếm, từ đó nâng cao đáng kể hiệu quả thuật toán.

Bản thân việc tìm điều kiện cần chính xác không có một quy tắc cố định. Điều đó đòi hỏi ta phải chịu khó suy nghĩ, biết quan sát phát hiện và thường xuyên tổng kết trong thực hành.

A Bài Puzzle

Có một bàn cờ ô vuông 5×5 . Trên bàn có 24 quân cờ khác nhau, lần lượt được ký hiệu bằng các chữ cái in hoa A, B, ..., X; còn một ô để trống, ký hiệu bằng dấu cách.

Mỗi bước của trò chơi di chuyển quân cờ ở phía trên, phía dưới, bên trái hoặc bên phải ô trống vào ô trống. Bốn thao tác này lần lượt được ký hiệu là 1, 2, 3, 4.

Nếu biết trạng thái ban đầu của bàn cờ và một dãy thao tác hữu hạn theo đúng thứ tự, ta nhận được duy nhất một trạng thái đích. Chẳng hạn, trong ví dụ của đề gốc, trạng thái đầu qua dãy thao tác 144223 sẽ đến trạng thái đích tương ứng. Tuy nhiên, thứ tự thao tác đúng ban đầu đã bị xáo trộn: nếu thực hiện dãy bị xáo trộn từ trạng thái đầu thì không nhận được trạng thái đích. Chỉ có thứ tự bị xáo trộn, còn số lượng từng loại thao tác không đổi.

Nhiệm vụ là tìm lại một dãy thao tác đúng ban đầu. Nếu có nhiều nghiệm, chỉ cần đưa ra một nghiệm bất kỳ.

Dữ liệu vào. Đọc từ tệp PUZZLE.DAT trong thư mục hiện tại. Năm dòng đầu là trạng thái đầu, mỗi dòng có năm ký tự. Năm dòng tiếp theo là trạng thái đích, mỗi dòng có năm ký tự. Dòng thứ mười một là số nguyên dương L , độ dài dãy thao tác ($L \leq 50$). Dòng thứ mười hai có L ký tự, biểu diễn dãy thao tác đã bị xáo trộn. Trong mỗi dòng không có ký tự thừa ở đầu hoặc cuối.

Dữ liệu ra. Ghi vào tệp PUZZLE.OUT. Nếu có nghiệm, in một dòng gồm L ký tự là dãy thao tác đúng ban đầu. Nếu vô nghiệm, in 0.

Ví dụ.

PUZZLE.DAT

TRGSJ

XDOKI

M VLN

WPABE

UQHCF

TRGSJ

XOKLI

MDVBN

WP AE

UQHCF

6

442231

PUZZLE.OUT

144223

B Bài Musketeers

Vào thời vua Louis XIII và vị hồng y Richelieu quyền lực, trong quán trọ Full Barrel, n chàng lính ngự lâm ăn xong bữa và đang uống rượu. Rượu chưa cạn, nên họ bắt đầu gây sự; một cuộc ẩu đả say xỉn nổ ra, trong đó mỗi người đều xúc phạm tất cả những người còn lại.

Một cuộc quyết đấu là không tránh khỏi. Nhưng ai đấu với ai, và theo thứ tự nào? Họ quyết định đứng thành vòng tròn và rút thăm theo thứ tự. Người được rút thăm sẽ đấu với người bên phải mình. Kẻ thua rời khỏi trò chơi, chính xác hơn là thi thể của người ấy được người hầu mang đi. Người kế tiếp đứng cạnh kẻ thua sẽ trở thành hàng xóm của người thắng.

Nhiều năm sau, khi các sử gia đọc hồi ký của người thắng cuộc, họ nhận ra rằng kết quả cuối cùng phụ thuộc rất lớn vào thứ tự các trận quyết đấu. Họ cũng biết từ luyện kiếm rằng ai có thể thắng ai trong từng cặp. Quan hệ “A thắng B” không có tính bắc cầu: có thể A giỏi hơn B, B giỏi hơn C, nhưng C lại giỏi hơn A. Vì vậy, trong ba người ấy, trận đầu tiên ảnh hưởng đến kết quả cuối cùng. Nếu A và B đấu trước thì cuối cùng C thắng; nếu B và C đấu trước thì cuối cùng A thắng. Các sử gia muốn xác định những người nào có thể sống sót.

Nhiệm vụ. n người đánh số liên tiếp từ 1 đến n đứng thành vòng tròn. Họ đấu $n - 1$ trận. Ở vòng đầu, một người nào đó, ví dụ số i , đấu với người bên phải mình, tức người số $i + 1$ hoặc số 1 nếu $i = n$. Người thua rời trò chơi và vòng tròn được khép lại để người kế tiếp trở thành hàng xóm của người thắng. Cho ma trận kết quả quyết đấu. Nếu $A_{i,j} = 1$ thì người i luôn thắng người j ; nếu $A_{i,j} = 0$ thì người i thua người j . Người k được xem là có thể thắng nếu tồn tại một thứ tự rút thăm gồm $n - 1$ trận để k thắng trận cuối.

Hãy đọc ma trận A từ tệp MUS.IN, tính các số hiệu người có thể thắng, rồi ghi chúng vào MUS.OUT.

Dữ liệu vào. Dòng đầu của MUS.IN chứa số nguyên n với $3 \leq n \leq 100$. Trong mỗi dòng của n dòng tiếp theo có một xâu gồm n chữ số 0 hoặc 1. Chữ số ở vị trí j của dòng i biểu diễn $A_{i,j}$. Với $i \neq j$, ta có $A_{i,j} = 1 - A_{j,i}$, và giả sử $A_{i,i} = 1$ với mọi i .

Dữ liệu ra. Dòng đầu của MUS.OUT ghi m , số người có thể thắng. Trong m dòng tiếp theo, ghi số hiệu của những người ấy theo thứ tự tăng dần, mỗi dòng một số.

Ví dụ.

```
Input
7
1111101
0101100
0111111
0001101
0000101
1101111
0100001
```

```
Output
3
1
3
6
```

Một thứ tự quyết đấu giúp người số 6 thắng là 1–2, 1–3, 5–6, 7–1, 4–6, 6–1. Chỉ còn hai người khác có thể thắng là 1 và 3.

C Bài Experiment

Gần đây, các nhà khoa học đã dùng tàu thăm dò để lấy lần lượt n_1 và n_2 mảnh thiên thạch có cấu trúc tương tự từ sao A và sao B . Cần thực hiện $n = \min(n_1, n_2)$ thí nghiệm hóa học để so sánh chúng; mỗi thí nghiệm lấy một mảnh từ sao A và một mảnh từ sao B . Do biến đổi hóa học trong thí nghiệm, một mảnh đã dùng thì không thể dùng lại. Vì đặc tính của loại thiên thạch này bị ảnh hưởng lớn bởi khối lượng, các nhà khoa học muốn chọn một phương án sao cho tổng trị tuyệt đối hiệu khối lượng của hai mảnh trong mỗi thí nghiệm là nhỏ nhất.

Hãy tính tổng nhỏ nhất ấy.

Dữ liệu vào. Tập vào là `Exp.dat`. Dòng đầu gồm hai số nguyên n_1, n_2 . Tiếp theo là n_1 dòng, mỗi dòng một số thực, biểu diễn khối lượng các mảnh từ sao A . Sau đó là n_2 dòng, mỗi dòng một số thực, biểu diễn khối lượng các mảnh từ sao B . Có $n_1, n_2 \leq 500$, và khối lượng mỗi mảnh không vượt quá 90.

Dữ liệu ra. Tập ra là `Exp.out`, gồm một số thực duy nhất là tổng nhỏ nhất cần tìm.

Ví dụ.

Input

4 2
44.9
50.0
77.2
86.4
59.8
58.9

Output

23.8